

Mongo DB

<https://www.youtube.com/watch?v=tw-w-gbNP0cA&t=624s>

- Mongosh
- Use database_name
- Show dbs

1. Creation of database and Collection

- > Use database_name
- > db.createCollection("myCollection")
- > show dbs
- > show collection

1. Crud Operation

1. Create

- . insertOne()
- . insertMany()

2. Read

- . find()
- . findOne()

3. Update

- . updateOne()
- . updateMany()

4. Delete()

- . deleteOne()
- . deleteMany()

1. Create()

// insertOne()

```
company>db.user.insertOne(  
{  
  "fname": "Amit",  
  "lname": "Kumar",  
  "age": 25,  
  "email": "amit@gmail.com",  
  "mobile_no": 965841223,  
  "address": {  
    "house_no": 42,  
    "nagar": "Saket",  
    "city": "Indore",  
    "state": "M.P.",  
    "pin_code": 452001  
  },  
  "hobbies": ["Cricket", "Football"]  
}  
)
```

Case 1: Inserting Single Document

```
company> db.user.insertOne(
... {
...   "fname": "Amit",
...   "lname": "Kumar",
...   "age": 25,
...   "email": "amit@gmail.com",
...   "mobile_no": 965841223
... }
... )
{
  acknowledged: true,
  insertedId: ObjectId('6999725204ccf058f7628ca2')
}
```

Case 2: Inserting Nested Document

```
company> db.user.insertOne(
... {
...   "fname": "Amit",
...   "lname": "Kumar",
...   "age": 25,
...   "email": "amit@gmail.com",
...   "mobile_no": 965841223,
...   "address": {
...     "house_no": 42,
...     "nagar": "Saket",
...     "city": "Indore",
...     "state": "M.P.",
...     "pin_code": 452001
...   }
... }
... )
{
  acknowledged: true,
  insertedId: ObjectId('6999741f04ccf058f7628ca3')
}
```

Case 3: Inserting Array

```
company> db.user.insertOne(
... {
...   "fname": "Amit",
...   "lname": "Kumar",
...   "age": 25,
...   "email": "amit@gmail.com",
...   "mobile_no": 965841223,
...   "address": {
...     "house_no": 42,
...     "nagar": "Saket",
...     "city": "Indore",
...     "state": "M.P.",
...     "pin_code": 452001
...   },
...   "hobbies": ["Cricket", "Football"]
... }
... )
{
  acknowledged: true,
  insertedId: ObjectId('699974d804ccf058f7628ca4')
}
```

//insertMany[{}, {}, {}] Like that

```
company>db.user.insertOne(  
[  
  {  
    "fname": "Sumit",  
    "lname": "Gupta",  
    "age": 20,  
    "email": "sumit@gmail.com",  
    "mobile_no": 9826433542,  
    "address": {  
      "house_no": 32,  
      "nagar": "Vijay",  
      "city": "Indore",  
      "state": "M.P.",  
      "pin_code": 452001  
    },  
    "hobbies": ["Chess", "Kabbadi"]  
  },  
  {  
    "fname": "Deepak",  
    "lname": "Mishra",  
    "age": 35,  
    "email": "deepak@gmail.com",  
    "mobile_no": 9977562389,  
    "address": {  
      "house_no": 142,  
      "nagar": "LIG",  
      "city": "Indore",  
      "state": "M.P.",  
      "pin_code": 452001  
    }  
  }  
]
```

```
    },  
    "hobbies": ["Cricket", "Football"]  
  },  
  {  
    "fname": "Vinit",  
    "lname": "Pal",  
    "age": 27,  
    "email": "vinit@gmail.com",  
    "mobile_no": 9425568923,  
    "address": {  
      "house_no": 36,  
      "nagar": "MIG",  
      "city": "Indore",  
      "state": "M.P.",  
      "pin_code": 452001  
    },  
    "hobbies": ["Cricket", "Boxing"]  
  }  
]  
)
```

2. Read()

1. find()

```
Company>db.user.find() //access all data
Company>db.user.find({hobbies:"Cricket"}) // Case Sensitive
company> db.user.find({"address.house_no":36})
company> db.users.findOne({ hobbies: "Cricket" }); //access single
```

3. Update()

1. updateOne()

//Update Existing Value

```
company> db.user.updateOne(
... {fname:"Amit"},
... {$set:{Age:25}}
... )
```

//Add a New Key and Value

```
company> db.user.updateOne(
... {fname:"Amit"},
... {$set:{Phone_No:07312761358}}
... )
```

//Add a New Value into Array

```
company> db.user.updateOne(
... {fname:"Amit"},
... {$push:{hobbies:"Running"}}
... )
```

//Extract a Value from Array

```
company> db.user.updateOne(  
... {fname:"Amit"},  
... {$pull:{hobbies:"Chess"}}  
... )
```

2. updateMany()

// Change done in Many Collections

```
company> db.user.updateMany(  
... {fname:"Amit"},  
... {$set:{Nationality:"Indian"}}  
... )
```

// Change done in Many Collections

```
company> db.user.updateMany(  
... {},  
... {$set:{Gender:"Male"}}  
... )
```

// Add a New Collections what match case not found

```
company> db.user.updateMany(  
    {fname:"Kamal"},  
    {$set:{Age:30, Mobile_No:9165841223, }},  
    {upsert:true}  
    )
```


4. Delete()

1. deleteOne()

Company>db.user.deleteOne({fname:"Kamal"}) // Case Sensitive

2. deleteMany()

Company>db.user.deleteMany({fname:"Kamal"}) // Case Sensitive

3. drop collection

Company>db.collection_name.drop()

Compony>db.order.drop()

4. drop database

Company>db.dropDatabase()

MongoDB Query Operators

There are many query operators that can be used to compare and reference document fields.

Comparison

The following operators can be used in queries to compare values:

- **\$eq:** Values are equal

```
company> db.user.find({age:{ $eq:20}})
```

- **\$ne:** Values are not equal

```
company> db.user.find({age:{ $ne:20}})
```

- **\$gt:** Value is greater than another value

```
company> db.user.find({age:{ $gt:30}})
```

- **\$gte:** Value is greater than or equal to another value

```
company> db.user.find({age:{ $gte:27}})
```

- **\$lt:** Value is less than another value

```
company> db.user.find({age:{ $lt:30}})
```

- **\$lte:** Value is less than or equal to another value

```
company> db.user.find({age:{ $lte:30}})
```

- **\$in:** Value is matched within an array

```
company> db.user.find({ age: { $in: [25, 30] } })
```

- **\$nin:** Value is matched within an array

```
company> db.user.find({ age: { $nin: [25, 30] } })
```

Logical

The following operators can logically compare multiple queries.

- **\$and:** Returns documents where both queries match

```
db.user.find({
  $or: [{age: 27}, {fname: "Vinit"}]
})
```

- **\$or:** Returns documents where either query matches

```
db.user.find({
  $or: [{age: 27}, {fname: "Vinit"}]
})
```

- **\$nor:** Returns documents where both queries fail to match

```
db.user.find({
  $nor: [{age: 27}, {fname: "Vinit"}]
})
```

- **\$not:** Returns documents where the query does not match

```
db.user.find({
  $not: [{age: 27}, {fname: "Vinit"}]
})
```

Evaluation

The following operators assist in evaluating documents.

- **\$regex:** Allows the use of regular expressions when evaluating field values
- **\$text:** Performs a text search
- **\$where:** Uses a JavaScript expression to match documents

MongoDB Aggregate

- **Aggregation** in MongoDB is used to **process data records and return computed results**.

It works like **GROUP BY in SQL** and allows operations such as **grouping, filtering, sorting, averaging, counting, etc.**

- The main command used is:
- `db.collection.aggregate([`
 `{ stage1 },`
 `{ stage2 },`
 `{ stage3 }`
 `])`
- Each stage performs an operation on the data.

1.Create a collection student

```
Test>show dbs
```

```
Test>use school
```

```
School>db.school.createCollection("students")
```

```
school> db.students.insertMany(  
[  
  { name: "Amit", age: 21, course: "BCA", marks: 75 },  
  { name: "Neha", age: 22, course: "BCA", marks: 85 },  
  { name: "Priya", age: 23, course: "MCA", marks: 90 },  
  { name: "Rahul", age: 21, course: "MCA", marks: 80 }  
]  
)
```

2. \$group (Grouping Documents) (Group students by course)

```
db.students.aggregate([
  {
    $group:{
      _id:"$course",
      totalStudents:{ $sum:1 }
    }
  }
])
```

Output:

```
{ "_id": "BCA", "totalStudents": 2 }
{ "_id": "MCA", "totalStudents": 2 }
```

3. \$match (Filtering Data) (Works like WHERE in SQL.)

```
db.students.aggregate([
  { $match: { course:"BCA" } }
])
```

Output:

```
school> db.students.aggregate([
...   { $match: { course:"BCA" } }
... ])
[
  {
    _id: ObjectId('69ad4831d0fb121ed7628ca0'),
    name: 'Amit',
    age: 21,
    course: 'BCA',
    marks: 75
  },
  {
    _id: ObjectId('69ad4831d0fb121ed7628ca1'),
    name: 'Neha',
    age: 22,
    course: 'BCA',
    marks: 85
  }
]
```

4. \$sort (Sorting) (Sort by marks.)

```
db.students.aggregate([
  { $sort:{ marks:-1 } }
])
```

1 → ascending

-1 → descending

```
school> db.students.aggregate([
...   { $sort:{ marks:-1 } }
... ])
[
  {
    _id: ObjectId('69ad4831d0fb121ed7628ca2'),
    name: 'Priya',
    age: 23,
    course: 'MCA',
    marks: 90
  },
  {
    _id: ObjectId('69ad4831d0fb121ed7628ca1'),
    name: 'Neha',
    age: 22,
    course: 'BCA',
    marks: 85
  },
  {
    _id: ObjectId('69ad4831d0fb121ed7628ca3'),
    name: 'Rahul',
    age: 21,
    course: 'MCA',
    marks: 80
  },
  {
    _id: ObjectId('69ad4831d0fb121ed7628ca0'),
    name: 'Amit',
    age: 21,
    course: 'BCA',
    marks: 75
  }
]
school> |
```

5. \$project (Selecting Fields) (Display only specific fields)

```
db.students.aggregate([
{
  $project:{
    name:1,
    marks:1,
    _id:0
  }
}]
```

Note: `_id: 0` means exclude the `_id` field from the output.

Output:

```
school> db.students.aggregate([
... {
...   $project:{
...     name:1,
...     marks:1,
...     _id:0
...   }
... }
... ])
[
  { name: 'Amit', marks: 75 },
  { name: 'Neha', marks: 85 },
  { name: 'Priya', marks: 90 },
  { name: 'Rahul', marks: 80 }
]
```

6. \$avg (Average Calculation) (Average marks of students)

```
db.students.aggregate([
{
  $group:{
    _id:null,
    averageMarks:{ $avg:"$marks" }
  }
}]
```

Note: `_id: null` means **do not group by any field** — treat all documents as one single group.

Output:

```
school> db.students.aggregate([
...   {
...     $group:{
...       _id:null,
...       averageMarks:{ $avg:"$marks" }
...     }
...   }
... ])
[ { _id: null, averageMarks: 82.5 } ]
school> |
```


7. \$max and \$min

```
db.students.aggregate([
  {
    $group:{
      _id:null,
      maxMarks:{ $max:"$marks" }
    }
  }
])
```

```
db.students.aggregate([
  {
    $group:{
      _id:null,
      minMarks:{ $min:"$marks" }
    }
  }
])
```

8. \$sum (Total)

```
db.students.aggregate([
  {
    $group:{
      _id:null,
      totalMarks:{ $sum:"$marks" }
    }
  }
])
```

9. \$limit (Limit results.)

```
db.students.aggregate([
  { $limit:2 }
])
```

Output:

```
school> db.students.aggregate([
... { $limit:2 }
... ])
[
  {
    _id: ObjectId('69ad4831d0fb121ed7628ca0'),
    name: 'Amit',
    age: 21,
    course: 'BCA',
    marks: 75
  },
  {
    _id: ObjectId('69ad4831d0fb121ed7628ca1'),
    name: 'Neha',
    age: 22,
    course: 'BCA',
    marks: 85
  }
]
school> |
```

10. \$count (Count documents.)

```
db.students.aggregate([
  { $count:"TotalStudents" }
])
```